Deep Learning Project Report

Meng Teck, Selina Tan Hui Shan, Lee Sean
{1007176, 1007741, 1006903}@mymail.sutd.edu.sg

1 Introduction

Financial fraud is a persistent challenge in banking systems, resulting in significant monetary losses each year. The solution, fraud detection, despite having much potential, still requires improvements to provide customers a better user experience. For our case, fraud detection makes for a particularly interesting case for testing anomaly detection — while it is reasonable to assume that the features that make up fraudulent transactions have underlying discrepancies from regular ones, this may not be true. Consider that fraud, especially sophisticated ones, are meant to evade detection often aiming to appear as normal transactions so as to not arouse suspicion. Hence, we aim to investigate how effective is coupling simple anomaly detection with binary classification in identifying cases of credit card fraud.

For this project, we will be attempting to take on the **IEEE-CIS Fraud detection dataset** — more on the dataset in the next section 2. Here, the project adopts a multi-phased modelling pipeline to detect fraudulent transactions under a constrained transaction-level setting, where limited historical and identity context is available[1]. The approach will integrate unsupervised representation learning, supervised classification and ensemble methods to progressively refine predictive performance under these constraints.

The pipeline will consist of four key phases. In the **first phase** autoencoders are trained on transaction features to test the presence of meaningful anomaly structures within the dataset. By learning to reconstruct non-fraudulent pattern. This autoencoder provides both latent representation and reconstruction errors which serve as candidate signals to potentially enrich downstream models.

The **second phase** will consist of training baseline supervised models using both neural and tree-based approaches to establish learned decision boundaries. Multi-layer Perceptrons (MLPs) are employed to capture smooth nonlinear interactions across features, hopefully, providing continuous approximations of the decision surface. While Gradient-Boosting Trees (CatBoost, XGBoost and LightGBM) will serve as the primary predictive backbone. Tree models are particularly well suited for tabular fraud detection tasks where feature relationships are often irregular, sparse and governed by some threshold-based rules. Boosted trees are strong at partitioning the feature space into finer regions allowing for sharp decision boundaries. Empirically, such models demonstrated strong performance in fraud detection and are a top choice for competitive tabular machine learning.

The **third phase** evaluates the integration of autoencoder-derived features into the supervised models. By augmenting the original feature space with anomaly signals. This will serve to access whether reconstruction-based representations provide any measurable improvements over baseline performance.

Finally, the **fourth phase** will involve combining all the strongest models found into a stacked ensemble framework. This meta-learning approach will take advantage of the complementary strengths of individual models by learning from their prediction outputs. The stacking mechanism exploits differences in model behavior and error patterns, enabling improved calibration and a more effective balance between fraud detection sensitivity and false positive control.

Overall, the pipeline reflects a progression from representation discovery to decision boundary construction and finally to meta-level error correction

[1]The models trained are constrained to only having access to the raw features of a single transaction

2 Dataset

The **IEEE-CIS Fraud detection dataset** (IEEE Computational Intelligence Society [IEEE-CIS], 2019) was chosen because it provides data coming from Vesta's real-world e-commerce transactions and contains a wide range of features, including transactional metadata, device information, anonymized raw and engineered variables, making it well-suited for studying fraud detection in realistic tabular settings. However, the raw data contains significant sparsity, irregularities, and missing values, requiring careful preprocessing before model training.

2.1 Feature Engineering and Cleaning

Raw transactional data often contains irregular time formats and missing values. To capture temporal patterns, the raw `TransactionDT` timestamp was decomposed into cyclical features: `TransactionHour`, `TransactionDay`, and `TransactionWeekday`. Moreover, handling missing data (NaN) requires careful consideration, particularly for Neural Networks. While a baseline approach might replace missing values with an extreme sentinel value (e.g., -999), this introduces artificial outliers that can heavily distort the gradient updates in deep learning models. To resolve this, our preprocessing pipeline isolates missingness as a feature of its own. For every numeric column, a binary missingness indicator (`_missing`) is generated. The original missing values are then imputed using the feature's median, followed by zero-mean, unit-variance standardization. Categorical variables are imputed with an 'Unknown' label and converted to category codes, ensuring the network can continuously learn without gradient disruption.

2.2 Autoencoder

To provide downstream classifiers with a stronger signal separating legitimate and fraudulent behavior, we integrated a Denoising Autoencoder into our preprocessing pipeline. The Autoencoder consists of a mirrored Multi-Layer Perceptron (MLP) structure utilizing Gaussian Error Linear Unit (GELU) activations, compressing the input into a dense, lower-dimensional latent space (z). Crucially, the Autoencoder is trained *exclusively* on non-fraudulent transactions. By learning the manifold of legitimate consumer behavior, transactions that deviate from the learned representation, such as fraudulent activity, tend to exhibit larger reconstruction errors. While this separation is not guaranteed, we can leverage this discrepancy by passing the entire dataset through the frozen Autoencoder and extracting a suite of anomaly signals:

- **Latent Representation (z):** The compressed embedding of the transaction.
- **Mean Squared Error (MSE):** The overall reconstruction loss, acting as a primary anomaly score.
- **Maximum & Top-K Errors:** The highest absolute reconstruction errors across features, isolating specific compromised variables.
- **L1 Error & Standard Deviation:** Metrics capturing the total volume and spread of the reconstruction discrepancies.

These extracted features are concatenated with the original standardized inputs, producing a highly enriched dataset for the ensemble and MLP classifiers.

This design is relevant due to the constraints of limited contextual information. where each transaction must be evaluated in isolation. By embedding the transactions into a learned representation of "non-anomalous" behaviour, the Autoencoder introduces additional structure that may not be explicitly captured by supervised models alone.

2.3 Data Splitting

The official test set provided by the IEEE-CIS Kaggle competition does not include ground truth labels, rendering it unusable for standard supervised evaluation. Consequently, we derived our own train, validation, and test splits entirely from the provided training set. Because financial fraud datasets are notoriously imbalanced, a random split could easily result in a validation or test set with zero fraudulent examples. To prevent this, the data splitting process utilizes stratified sampling. This ensures that the baseline ratio of fraud to non-fraud transactions is strictly preserved across the training, validation (10%), and test (10%) subsets.

3 Model Training and Methodology

This section outlines the experimental methodology used to develop and evaluate the proposed fraud detection pipeline. The training process is structured into the four distinct phases mentioned in section 1, each designed to test a specific hypothesis about the data and model behavior under a constrained transaction-level setting.

Given the highly imbalanced nature of fraud detection, model performance is primarily evaluated using recall-oriented metrics. As in this case, standard accuracy is a misleading metric, since a model could achieve over 96% accuracy simply by predicting that no transactions are fraudulent. Instead, primary benchmarks we used were the F2 score, which places greater emphasis on minimizing false negatives. Additional metrics such as precision, F1 score, and ROC-AUC are also reported to provide a comprehensive view of model performance. We prioritize the F2-score because it places a higher mathematical weight on recall than precision, accurately reflecting the business reality that the financial cost of missing a fraudulent transaction (a false negative) is significantly higher than the minor inconvenience of a false alarm (a false positive).

Each phase progressively builds upon the previous one, moving from unsupervised representation learning to supervised classification. The fourth phase on ensembling will be covered in the next section 4 as the ensemble fully depends on the previous individually trained models.

3.1 Phase 1: Unsupervised Representation learning via Autoencoder

The first phase examines whether meaningful anomaly structures exists within the feature space. A Denoising Autoencoder is trained exclusively on non-fraudulent transactions. Enabling it to learn a compact representation of typical user behavior.

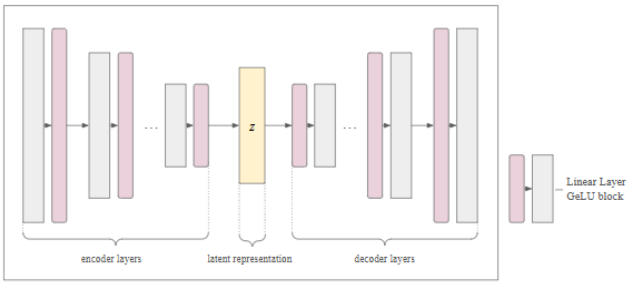


Figure 1: Visualisation of simple autoencoder arhcitecture.

The model is optimised using reconstruction loss (MSE), with gaussian noise injected during training to improve robustness. performance is evaluated by comparing reconstruction error distribution between fraudulent and non-fraudulent samples on the held out data (validation and test set). Below are some of the training statistics gathered during model training.

Metric	**Normal Transaction**	**Fraudulent Transactions**
Mean Reconstruction Error	0.101	0.830
Error Ratio (Fraud / Normal)	-	8.21×

Table 1: Mean reconstruction error for normal and fraudulent transactions on the test set

As shown in Table 1, fraudulent transactions exhibit a substantially higher mean reconstruction error compared to normal transactions. Specifically, normal transactions have a mean error of 0.101, while fraudulent transactions have a significantly larger error of 0.830.

This results in an error ratio of approximately 8.21×, indicating strong separation between the two classes in the learned representation space.

This suggests that the autoencoder has effectively learned to reconstruct normal transaction patterns, while failing to accurately reconstruct anomalous (fraudulent) samples. Consequently, reconstruction error serves as a reliable anomaly score for fraud detection.

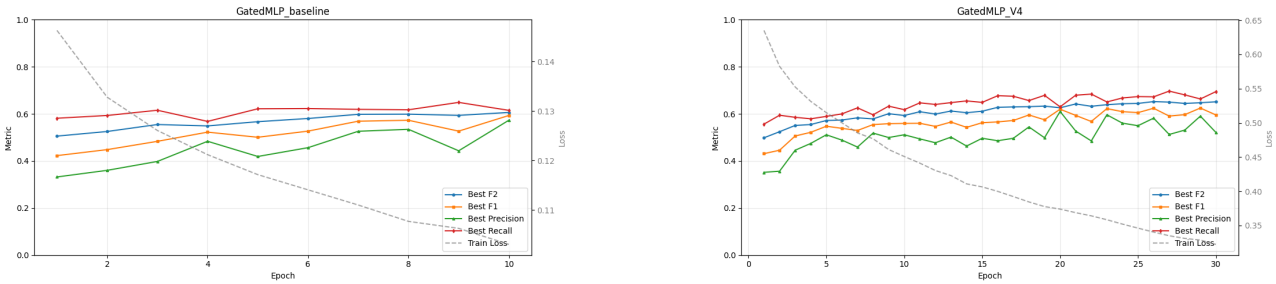
3.2 Phase 2: Baseline Supervised Models

The second phase establishes baseline classifiers using both neural and tree-based approaches. Gated MLPs are trained to model smooth nonlinear feature interactions, while Gradient Boosting Decision Trees serve as the primary predictive backbone due to their effectiveness on tabular data.

Model training evaluation is conducted using a validation set with threshold-based metrics. Rather than relying on a fixed decision threshold, a threshold sweep is performed to identify the optimal classification boundary. The F2 score is used as the primary optimisation metric, as it places greater emphasis on recall and penalizes false negatives more heavily than F1. This ensures that the models are tuned to prioritise fraud detection sensitivity while maintaining reasonable precision as opposed to a full recall heavy optimisation.

GatedMLP training: For neural models, training is performed using the AdamW optimizer with binary cross-entropy loss (`BCEWithLogitsLoss`). To account for the severe class imbalance inherent in fraud detection, a positive class weighting (`pos_weight`) is incorporated into the loss function, increasing the penalty for misclassified fraudulent transactions.

Below, we can observe two baseline Gated MLP configurations being trained, differing in hidden layer dimensionality. These curves for loss and validation metrics[2] provide insight into convergence behavior, generalisation performance, and the trade-offs between recall and precision during training.



(a) GatedMLP baseline, hidden_dims = {256, 128, 64}

(b) GatedMLP baseline, hidden_dims = {512, 256, 128, 64}

Figure 2: Training curves for two baseline GatedMLP models trained for 30 epochs

Interpretation of Figs 2a and 2b: Both models show stable convergence with improving validation metrics. The smaller MLP saturates early, indicating limited capacity, while the larger model achieves higher recall and F2 over more epochs but with greater variability. This suggests increased capacity improves fraud detection sensitivity, but with reduced stability. As such we also tested larger hidden dimensions over longer epochs for an experiment which can be seen in the figures below:

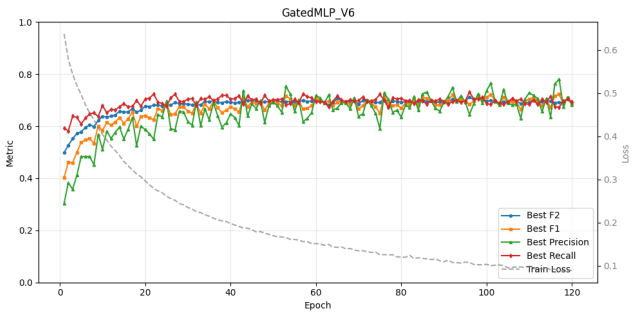


Figure 3: GatedMLP baseline, hidden_dims = {1024, 512, 256, 128, 64}, 120 epochs

Interpretation of Fig 3: Increasing model capacity and training duration yields improvements, with F2 and recall stabilising at higher levels (around 0.69 and 0.75 respectively). Precision variability persists.

[2]During training and considering which model to save, we chose to display the validation metrics at 0.5 threshold. Although we also track the best possible threshold for optimising F2 score.

The final validation metrics for the baseline MLPs can be found in the tables below:

Model	Precision	Recall	F1	F2	Accuracy	best threshold
GatedMLP Baseline	0.5730	0.6149	0.5932	0.6060	0.9705	0.1680
GatedMLP V2	0.5815	0.6662	0.6210	0.6473	0.9715	0.6117
GatedMLP V6	0.6448	0.7315	0.6854	0.7123	0.9765	0.7119

Table 2: Validation performance of Gated MLP models at optimal thresholds (F2-optimised) on validation set

Model	Threshold	Precision	Recall	F1	F2	Accuracy
GatedMLP Baseline	0.5	0.8513	0.4819	0.6154	0.5277	0.9789
	Opt	0.5730	0.6149	0.5932	0.6060	0.9705
GatedMLP V2	0.5	0.4483	0.7054	0.5482	0.6328	0.9593
	Opt	0.5815	0.6662	0.6210	0.6473	0.9715
GatedMLP V6	0.5	0.5194	0.7576	0.6163	0.6940	0.9670
	Opt	0.6448	0.7315	0.6854	0.7123	0.9765

Table 3: Comparison of validation performance at default (0.5) and optimal (F2-optimised) thresholds on validation set

Observation for table 3: Across all models, threshold optimsation[3] consistently improves F2 score by rebalancing precision and recall. At the default threshold of 0.5, models exhibit varying biases ranging from conservative behavior (high precision, low recall) to more aggressive detection (high recall, lower precision). Adjusting the threshold allows the model to better align with the task objective by shifting this trade-off.

Training Decision Trees: Tree-based models are trained using their respective gradient boosting frameworks[4], following standard supervised learning procedures. These models serve as a strong baseline due to their ability to capture complex, non-linear feature interactions in tabular data.

Unlike neural models, gradient boosting tree do not follow a traditional epoch-based training process. Instead, they iteratively build ensembles of decision trees, where subsequent trees corrects the errors of it's predecessors. As a result, training evaluation is instead conducted using final validation performance metrics.

Model	Evaluation Metric (PR-AUC)	Best Iteration	Depth	Learning Rate
CatBoost without AutoEncoder	0.6547	999	6	0.05
XGBoost without AutoEncoder	0.8234	4999	8	0.025

Table 4: Comparison of evaluation metrics without AutoEncoder on validation set

However, it is important to note that threshold tuning does not change the underlying ranking performance of the model, and **does not affect metrics such as PR-AUC or ROC-AUC**. Instead, it serves as a post-hoc decision that selects an appropriate operating point based on the desired balance between false positives and false negatives. Hence, threshold optimisation is used to **tailor model behavior to fraud detection**, rather than measure intrinsic model quality.

[3]"Optimal" here does not refer to an objective "best" value, instead it represents the chosen optimisation metric being targeted — F2 score

[4]XGBoost and CatBoost python libraries

3.3 Phase 3: Training models with Pre-trained Autoencoder

In the third phase, the pre-trained Autoencoder is integrated directly into the architecture of the supervised models, rather than being used as a static preprocessing step. Specifically, the encoder component is incorporated as a frozen module within the Gated MLP, where it operates as a fixed feature transformation layer during the forward pass.

Given a feature x, the frozen Autoencoder produces both a latent representation z and reconstruction-error based features. These outputs are concatenated with the original input features dynamically within the model, forming an augmented representation which is processed by subsequent layers.

This design ensures that all feature transformations remain encapsulated within the model itself, maintaining consistency across training and evaluation pipelines. It also avoids discrepancies that may arise from external preprocessing steps (Especially in group development settings).

Training of the augmented models follows the same optimisation framework as in Phase 2. The models are optimised using the `AdamW optimizer` with `BCEWithLogitsLoss`, incorporating positive class weighting to address class imbalance. Evaluation is again performed using threshold sweeping on the validation set, with the F2 score used as the primary selection criterion.

The objective of this phase is to assess whether embedding anomaly-aware representations directly within the model architecture provides measurable improvements over baseline models trained solely on raw features.

Training GatedMLP with Autoencoder: The notation AE16 and AE256 refers to the dimensionality of the latent representation, where the encoder compresses the input into a 16-dimensional or 256-dimensional latent space respectively. This allows us to evaluate the impact of representation capacity on downstream classification performance.

Two variants of reconstruction-based features are explored. The first uses a scalar reconstruction error, computed as the mean squared error across all input features, providing a compact anomaly score relative to learned normal behavior.

$$\text{scalar_reconstruction_err} = \frac{1}{x_\text{dim}} \sum^{x_\text{dim}} (x_{\text{reconstructed}} - x)^2$$

The second, denoted as *ReconVec*, uses the full reconstruction error vector, retaining per-feature reconstruction discrepancies. While ReconVec preserves more detailed information, it significantly increases input dimensionality and may introduce noise, as individual feature-level errors are not always informative for classification.

Model	**Precision**	**Recall**	**F1**	**F2**	**Accuracy**
GatedMLP V6 (Baseline)	0.6448	0.7315	0.6854	**0.7123**	0.9765
GatedMLP + AE16	**0.6680**	0.7088	**0.6878**	0.7002	**0.9775**
GatedMLP + AE256	0.6330	0.6952	0.6627	0.6818	0.9752
GatedMLP + AE16 (ReconVec)	0.6041	0.7034	0.6500	0.6810	0.9735

Table 5: Comparison of baseline and Autoencoder-augmented MLP models at optimal thresholds on validation set

Observation for Table 5: When compared to the strongest baseline (GatedMLP V6), Autoencoder-augmented models do not outperform in terms of F2 score, with the baseline achieving the highest overall value. However, the AE16 model demonstrates improved precision and slightly higher F1 score, indicating a shift in the precision–recall trade-off. Suggesting that while Autoencoder features do not directly improve overall performance under the chosen metric, they alter the model's decision behavior in a meaningful way.

These differences indicate that the models are capturing distinct aspects of the data, rather than converging to identical decision boundaries. As such, raw performance metrics alone may not fully reflect their complementary strengths, motivating further analysis of model disagreement and ensemble potential in the subsequent section 4.

We also note the differences in using the Autoencoder for the trees compared to the baseline without the Autoencoder shown below.

Model	**Evaluation Metric (PR-AUC)**	**Best Iteration**	**Depth**	**Learning Rate**
CatBoost without AutoEncoder	0.6547	999	6	0.05
XGBoost without AutoEncoder	0.8234	4999	8	0.025
CatBoost with AutoEncoder	0.7878	2508	10	0.03
XGBoost with AutoEncoder	0.8165	4588	10	0.025

Table 6: Comparison of evaluation metrics with and without AutoEncoder on validation set

The results reveal that the inclusion of an AutoEncoder does not consistently improve the performance of tree-based models. While CatBoost shows a substantial increase in PR-AUC from 0.6547 to 0.7878 when combined with AutoEncoder-generated features, XGBoost experiences a slight decrease from 0.8234 to 0.8165. Notably, the best overall single-model performance is achieved by XGBoost without AutoEncoder, indicating that additional feature learning does not universally benefit tree-based methods. This suggests that gradient boosting models are already capable of capturing complex feature interactions directly from the original feature space, and that AutoEncoder compression may sometimes lead to loss of discriminative information.

Despite this, AutoEncoder-based representations remain valuable within an ensemble framework. The final ensemble incorporates multiple models, including Gated MLP variants trained on both raw and AutoEncoder-derived features, as well as CatBoost, XGBoost models using AutoEncoder inputs. This design introduces diversity in the learned representations, models trained on raw features capture fine-grained patterns, while AutoEncoder-based models provide compressed and potentially denoised views of the data. As a result, even though AutoEncoder features do not consistently improve individual model performance, they enhance ensemble robustness by reducing model correlation and improving generalization. This highlights that representation learning should be evaluated not only at the level of standalone models, but also in terms of its contribution to ensemble diversity.

4 Model Behaviour Analysis and Ensemble Motivation

Building on the observations from section 3, we note that models with similar aggregate performance metrics can exhibit fundamentally different decision behaviors. Such as variations in precision–recall tradeoffs, threshold sensitivity which suggests that different models capture distinct aspects of the underlying data distribution.

There is a limitation of evaluating models solely based on single metrics such as F2/F1 score. While such metrics are useful for benchmarking, they hide differences in model calibration and error profiles.

To address this, we evaluate models on the held out test set using both threshold-dependent metrics (e.g., F2, Recall, Precision) and threshold-independent metrics (e.g., PR-AUC), and analyse their complementary strengths. This enables us to construct an ensemble that leverages diverse decision boundaries rather than relying on a single best-performing model. Below is the summary evaluation results of the best models on the held out test set.

Model	**F2**	**PR-AUC**	**Precision**	**Recall**	**F1**
Gated MLP Base (V6)	0.6651	0.7103	0.6116	0.6800	0.6440
Gated MLP AE16	0.6741	0.7142	0.6213	0.6887	0.6533
Gated MLP AE256	0.6425	0.6879	0.6024	0.6534	0.6268
LightGBM	0.6967	0.7414	0.5907	0.7294	0.6528
CatBoost	0.7218	0.7685	**0.6665**	0.7371	0.7000
XGBoost	**0.7382**	**0.8019**	0.6554	**0.7623**	**0.7048**

Table 7: Performance metrics of evaluated models on the test set. Both threshold-dependent (F2, Recall) and threshold-independent (PR-AUC) metrics are reported.

Observation of table 7 The results show that tree-based models clearly outperform neural models across all primary metrics, including F2, PR-AUC, precision, and recall, with XGBoost achieving the strongest overall performance. Among the neural approaches, only the baseline Gated MLP (V6) and the AE-16 variant remain somewhat competitive, though still falling behind tree-based methods.

However, this does not present the full picture for model selection. While tree models achieve superior aggregate performance, the neural models seem to exhibit different precision–recall trade-offs and decision characteristics. This suggests that they are capturing distinct patterns within the data rather than simply underperforming.

As such, aggregate metrics alone are insufficient to explain these behavioural differences. To better understand this, we first examine each model individually in the following subsections, before returning to a comparative behavioural analysis using disagreement metrics.

4.1 Gated MLP

Traditional Multi-Layer Perceptrons (MLPs) often struggle to achieve state-of-the-art performance on tabular data compared to tree-based models. To bridge this gap, we implemented a custom Gated MLP (`FraudMLP`). Instead of standard dense layers, this model utilizes `GatedLayer` blocks, where the linear projection is split in half: one half passes through a GELU activation, and the other passes through a Sigmoid gate. This multiplicative gating mechanism allows the network to dynamically filter out noisy features and learn more complex, non-linear interactions within the transaction data. Furthermore, the Gated MLP was explicitly designed to ingest the concatenated raw features alongside the latent representation (z) and reconstruction error vectors generated by our pre-trained Autoencoder, allowing it to directly optimize its weights based on the anomaly signals.

4.1.1 Gated MLP with AE-256

To further enhance feature representation, we extend the base Gated MLP by integrating an AutoEncoder with a 256-dimensional latent space (AE-256) trained within our pipeline. In this configuration, the AutoEncoder is trained in a semi-supervised manner using only non-fraudulent transactions to learn a compressed representation of normal transaction behaviour.

Specifically, the AutoEncoder maps high-dimensional transaction features into a latent representation ($z \in \mathbb{R}^{256}$), which captures compact and informative representations of legitimate transaction patterns. This latent

embedding is then concatenated with the original input features and passed into the Gated MLP. The AutoEncoder is frozen during Gated MLP training (`freeze_encoder=True`) to preserve the learned representation of normal behaviour and ensure stable anomaly-aware feature extraction.

By incorporating the AE-256 latent representation, the model benefits from both raw feature interactions and learned structural representations of normal transactions, improving its ability to distinguish fraudulent transactions from legitimate ones.

The reconstruction error distribution for fraudulent and non-fraudulent transactions is shown in Figure 4.

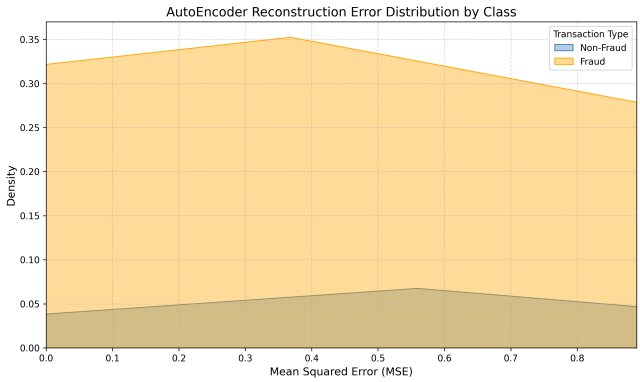


Figure 4: AutoEncoder Reconstruction Error Distribution for Fraud vs Non-Fraud Transactions

The results show a clear separation between the two classes.

- **Non-Fraud (Legitimate transactions):** These exhibit lower and more concentrated reconstruction errors, indicating that the AutoEncoder is able to effectively reconstruct patterns similar to those seen during training.
- **Fraudulent transactions:** These exhibit significantly higher reconstruction errors, reflecting the model's inability to accurately reconstruct anomalous patterns that deviate from normal behaviour.

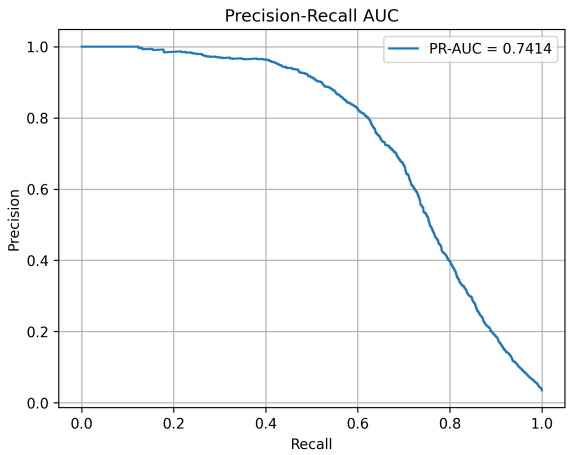
This separation confirms that reconstruction error is a strong anomaly signal. It is subsequently used as part of the feature representation passed to downstream classifiers such as LightGBM, CatBoost and XGBoost, improving their ability to distinguish fraudulent from legitimate transactions.

4.2 LightGBM

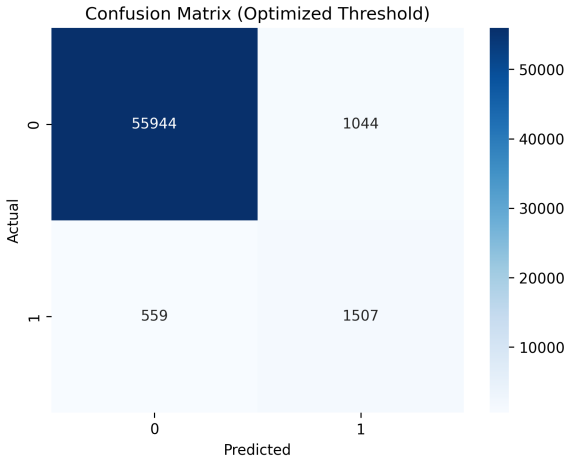
LightGBM (Light Gradient Boosting Machine) was selected as our first tree-based baseline, because it serves as a fast and stable learner, achieving an F2 score of 0.697 and providing a strong high-recall foundation for downstream ensembling. However, it was not used in our final ensemble because it did not provide much benefits in combination with our other stronger tree-based models.

To evaluate its performance on the imbalanced fraud detection task, we first examine the Precision–Recall (PR) curve. As shown in Figure 5, the model achieves a PR-AUC of 0.7414, indicating strong discriminative ability on the minority fraud class. The curve shows that precision remains close to 1.0 at lower recall levels, suggesting that high-confidence predictions are highly reliable. As recall increases beyond approximately 0.6, precision gradually declines, reflecting the expected trade-off as the model captures more ambiguous fraud cases.

While the PR curve captures ranking performance across thresholds, the confusion matrix provides a complementary view of classification performance at a fixed operating point. Using the F_2-optimised threshold of 0.615, the confusion matrix (Figure 5) shows that the model correctly identifies 1,507 fraudulent transactions while maintaining a reasonable balance between false positives and false negatives. This highlights the model's effectiveness in prioritising recall, which is critical in fraud detection scenarios.



(a) Precision–Recall curve (PR-AUC = 0.7414)



(b) Confusion matrix at F2-optimised threshold (0.615)

Figure 5: LightGBM performance: (left) ranking performance across thresholds, and (right) classification performance at the selected operating point.

To determine the optimal operating point, we analyse the relationship between the classification threshold and the F_2-score, as shown in Figure 6. The curve exhibits a clear peak at approximately 0.615. Lower thresholds lead to reduced precision due to an excess of false positives, while higher thresholds reduce recall by missing fraudulent transactions.

Selecting the threshold corresponding to this peak ensures that the model is tuned for recall-oriented performance, aligning with the objective of maximising fraud detection. This further demonstrates that the default threshold of 0.5 is suboptimal for imbalanced classification, and that explicit threshold tuning is essential for operational deployment.

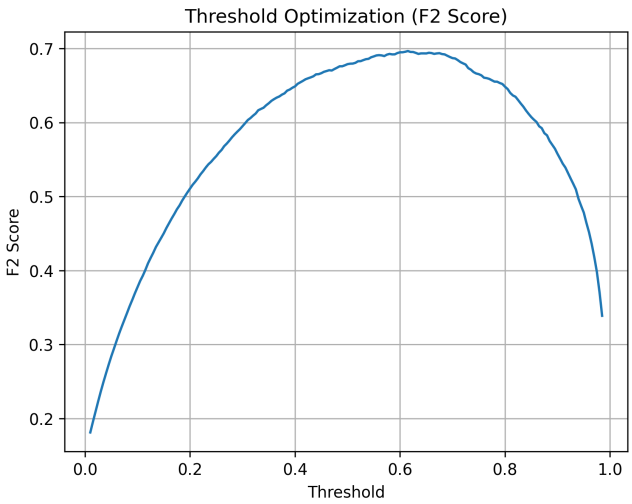


Figure 6: F_2-score across classification thresholds for the LightGBM model

4.3 CatBoost

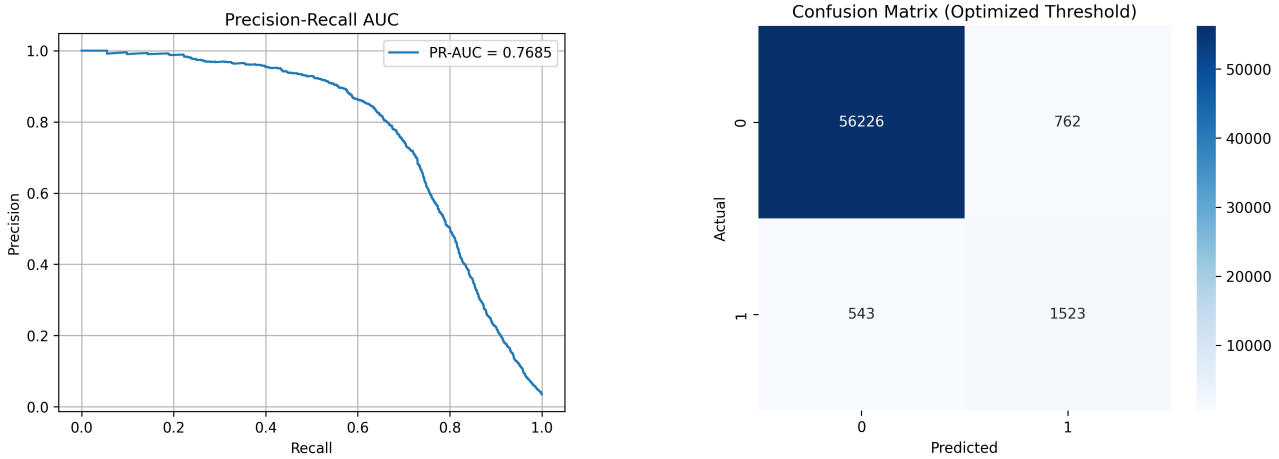
CatBoost (Categorical Boosting) was integrated into our architecture for its strong ability to handle complex, heterogeneous data distributions and its robustness against overfitting. Although our preprocessing pipeline converts categorical variables into numeric encodings, CatBoost's ordered boosting mechanism is specifically designed to reduce target leakage, a common issue in traditional gradient boosting methods.

We extend CatBoost using a custom `CatBoostAEWrapper`, which integrates Autoencoder-derived anomaly features, including reconstruction error statistics such as MSE, maximum error, and standard deviation of errors. By incorporating these explicit anomaly signals into the decision process, CatBoost is able to better separate

fraudulent and legitimate transactions. In this configuration, the model achieves an F2 score of 0.7218, making it a strong candidate for ensemble stacking.

To evaluate its performance on the imbalanced fraud detection task, we first examine the Precision–Recall (PR) curve. As shown in Figure 7, CatBoost achieves a PR-AUC of 0.7685, outperforming LightGBM and demonstrating improved capability in identifying minority-class (fraudulent) transactions. The curve indicates strong precision at lower recall levels, with a gradual decline as recall increases, reflecting the expected trade-off in capturing more ambiguous fraud cases.

While the PR curve captures ranking performance across all thresholds, the confusion matrix provides a complementary view of classification performance at a fixed operating point. Using the F2-optimised threshold of 0.57, the confusion matrix shows that CatBoost correctly identifies a higher number of fraudulent transactions while maintaining a comparable false positive rate. This highlights its effectiveness in improving recall without substantially sacrificing precision.



(a) Precision–Recall curve (PR-AUC = 0.7685)

(b) Confusion matrix at F_2-optimised threshold (0.57)

Figure 7: CatBoost performance: ranking performance via PR curve and classification performance at the optimal decision threshold.

To determine the optimal operating point, we analyse the relationship between the classification threshold and the F_2-score, as shown in Figure 8. The curve peaks at approximately 0.57, indicating the best balance between precision and recall for this model. Below this threshold, performance degrades due to an increase in false positives, while above it, recall drops as more fraudulent transactions are missed.

The fact that CatBoost's optimal threshold (0.57) is lower than that of LightGBM (0.615) suggests that its predicted probabilities are better calibrated toward the positive (fraud) class under the same feature space. This further reinforces its suitability for imbalanced classification tasks where recall is prioritised.

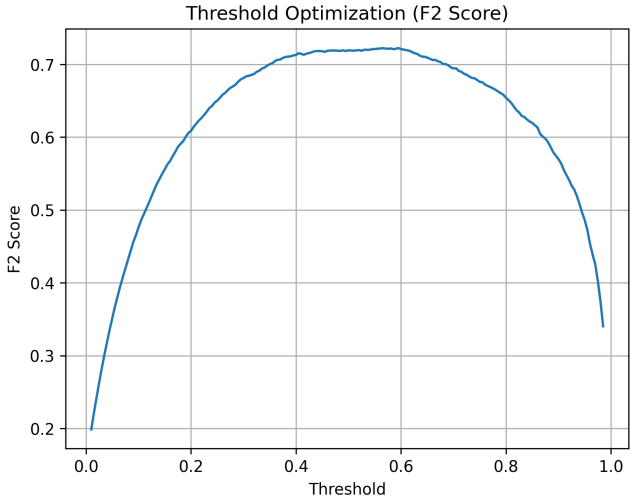


Figure 8: F_2-score across classification thresholds for the CatBoost model

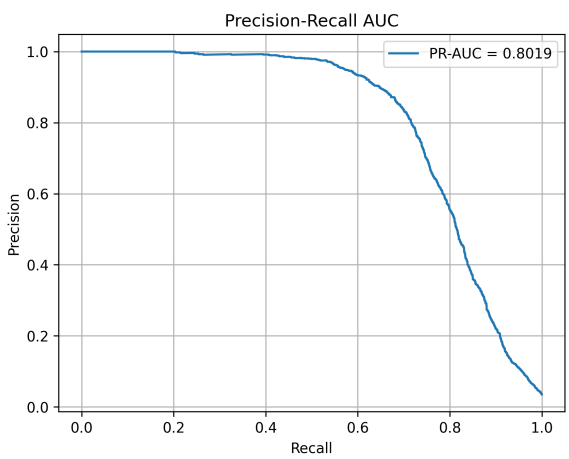
4.4 XGBoost

XGBoost (eXtreme Gradient Boosting) was selected for its strong regularization capabilities and efficient second-order gradient optimisation. Its use of both L1 and L2 regularisation on leaf weights, combined with sparsity-aware split finding, makes it particularly well-suited for fraud detection tasks characterised by rare and highly imbalanced events.

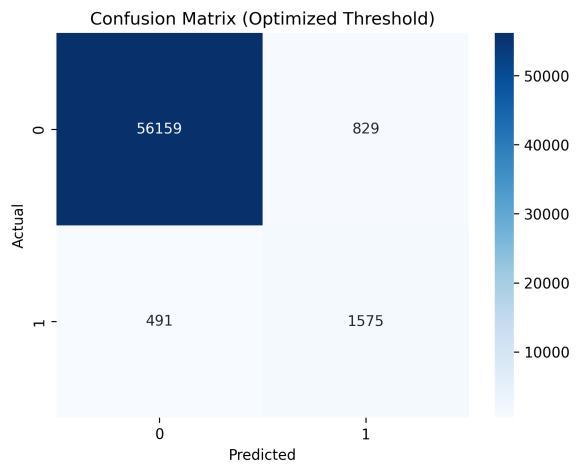
In our experiments, XGBoost emerged as the strongest standalone model, achieving an F2 score of 0.7382 and a PR-AUC of 0.8019. Its strong recall (0.7623) combined with high precision (0.6554) enabled it to effectively identify fraudulent transactions while maintaining a reasonable false positive rate, making it a key contributor to the final stacked ensemble.

To evaluate its performance under class imbalance, we first examine the Precision–Recall (PR) curve. As shown in Figure 11, XGBoost achieves the highest PR-AUC among all tree-based models, indicating superior discriminative ability for the minority fraud class. The curve demonstrates consistently strong precision at moderate recall levels, with a gradual decline as recall increases, reflecting the expected trade-off in capturing more difficult fraud cases.

While the PR curve evaluates ranking performance across thresholds, the confusion matrix provides a complementary view of classification performance at a fixed operating point. Using the F2-optimised threshold of 0.095, the confusion matrix shows strong fraud detection capability, with a high number of correctly identified fraudulent transactions and controlled false positives. This confirms XGBoost's effectiveness as the best-performing standalone model in this study.



(a) Precision–Recall curve (PR-AUC = 0.8019)



(b) Confusion matrix at F2-optimised threshold (0.095)

Figure 9: XGBoost performance: ranking performance via PR curve and classification performance at the optimal decision threshold.

To determine the optimal operating point, we analyse the relationship between classification threshold and the F2 score, as shown in Figure 10. The curve peaks at approximately 0.095, indicating the best trade-off between precision and recall for this model. Below this threshold, false positives increase sharply, while above it, recall drops due to missed fraudulent transactions.

Notably, the optimal threshold for XGBoost is significantly lower than that of LightGBM (0.615) and CatBoost (0.57), reflecting differences in probability calibration and decision boundary sharpness across tree-based methods. This further highlights the importance of model-specific threshold tuning in imbalanced classification settings.

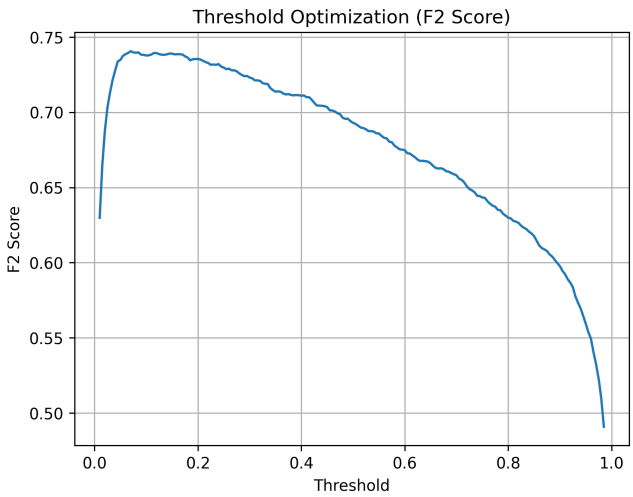


Figure 10: F2 score across classification thresholds for the XGBoost model

Having analysed the individual characteristics of each model, we now return to a comparative perspective. While several models achieve similar aggregate performance, they differ in how they make predictions. To quantify these differences, we introduce a simple heuristic metric termed the **Disagreement Rate ($\mathbf{D_{Rate}}$)**.

$$D_{\text{Rate}}(f_i, f_j) = \frac{1}{N} \sum_{k=1}^{N} \mathbf{1}\big(\hat{y}_k^{(i)} \neq \hat{y}_k^{(j)}\big)$$

Where $\hat{y}_k^{(i)}$ and $\hat{y}_k^{(j)}$ denote the predicted class labels of models f_i and f_j for sample k, and $\mathbf{1}(\cdot)$ is the indicator function.

Intuitively, $\mathbf{D}_{\text{Rate}}$ measures how often two models disagree on their predictions. A low disagreement rate indicates that models behave similarly and are likely capturing overlapping patterns in the data, while a high disagreement rate suggests that the models are learning different decision boundaries.

This metric is particularly useful in the context of ensemble learning, where diversity among base learners is a key factor in improving overall performance. Models with higher disagreement rates are more likely to contribute complementary information, making them stronger candidates for inclusion in a stacked ensemble.

Below are the $\mathbf{D}_{\text{Rate}}$ between the three neural models (Gated MLP Base (V6), Gated MLP AE16, Gated MLP AE256) using the test dataset, along with the defnitions for some the terms we are using.

Fraud Rate: For each agreement category, we compute the **fraud rate**, defined as the proportion of samples within that category that are truly fraudulent (i.e., the mean of the ground truth labels). This provides a measure of how reliable a given level of model agreement is as an indicator of fraud.

n-way Agreement and Disagreement: Given a set of M models, we define **n-way agreement:** as the subset of samples for which exactly n models predict the positive class (fraud). With three models ($M = 3$), this results in four categories: 0-way, 1-way, 2-way, and 3-way agreement[5].

Conversely, **disagreement** measures whether model predictions differ for a given sample. Pairwise disagreement ($\mathbf{D}_{\text{Rate}}$) captures the proportion of samples where two models produce different predictions, while multi-model disagreement reflects the proportion of samples where not all models agree. These definitions allow us to quantify both consensus (agreement) and diversity (disagreement) in model predictions.

Metric	**Value**
V6 vs AE16 $\mathbf{D}_{\text{Rate}}$	0.0207
V6 vs AE256 $\mathbf{D}_{\text{Rate}}$	0.0227
AE16 vs AE256 $\mathbf{D}_{\text{Rate}}$	0.0185
3-way $\mathbf{D}_{\text{Rate}}$	0.0309
3-way Fraud Rate	0.8241
2-way Fraud Rate	0.2990
1-way Fraud Rate	0.1325

Table 8: Disagreement and fraud concentration

Agreement Category	**Proportion**
3-way Agreement	0.0253
2-way Agreement	0.0087
1-way Agreement	0.0222
0-way Agreement	0.9437

Table 9: Agreement distribution

Disagreement Analysis (Table 8) While the neural models exhibit similar aggregate performance, disagreement analysis reveals meaningful differences in their decision behaviour. Pairwise disagreement rates of approximately 2% indicate that the models produce different predictions on a non-trivial subset of samples.

While this may appear small, it is significant given the scale of the dataset and the highly imbalanced nature of fraud detection. Even a 2% disagreement corresponds to thousands of transactions where predictions diverge. These regions are particularly important, as they often correspond to ambiguous cases where additional modeling capacity can improve classification.

The presence of disagreement confirms that the models are not redundant, but instead capture different aspects of the underlying data distribution, leading to complementary predictive behaviour.

Agreement Analysis (Table 9) The agreement distribution shows that the majority of samples fall into the 0-way (all models predicted non-fraud) agreement category, reflecting the highly imbalanced nature of the dataset where most transactions are legitimate. However, the fraud rate increases sharply with the level of agreement.

In particular, samples with 3-way agreement exhibit a fraud rate of over 82%, indicating that when all models independently predict fraud, the prediction is highly reliable. In contrast, samples with only partial agreement (1-way or 2-way) show substantially lower fraud rates, suggesting increased uncertainty in these regions.

These results demonstrate that model agreement acts as a strong proxy for prediction confidence, while disagreement highlights difficult or ambiguous cases where models rely on different decision boundaries.

This structured pattern of agreement and disagreement suggests that ensemble methods can leverage consensus for high-confidence predictions while using disagreement to refine decisions in uncertain regions.

[5]"n-way agreement" denotes how many models predict a sample as fraudulent. For example, 3-way agreement means all models predict fraud, while 0-way agreement means all models predict non-fraud. Intermediate cases (1-way, 2-way) indicate partial agreement among models.

Similarly we also performed a quick $\mathbf{D}_{\text{Rate}}$ anaylsis on the Tree Models, found below:

Metric	**Value**
CatBoost vs XGBoost D_{Rate}	0.0119
Both predict Fraud	1994
Both predict Non-Fraud	56359
CatBoost only	291
XGBoost only	410
Cat-only Fraud Rate	0.1031
XGB-only Fraud Rate	0.2000
Both-Fraud Rate	0.7487
Either-Fraud Rate	0.5955

Table 10: Disagreement and fraud concentration for tree-based models

Disagreement Analysis (Tree Models) Tree-based models exhibit a lower disagreement rate (approximately 1.2%) compared to neural models, indicating more consistent predictions. However, the disagreement that does occur remains informative. Samples where both models predict fraud show a high fraud rate of approximately 75%, reinforcing that agreement corresponds to high-confidence predictions.

Interestingly, disagreement regions reveal asymmetry between models. XGBoost-only predictions exhibit a higher fraud rate (20.0%) compared to CatBoost-only predictions (10.3%), suggesting that XGBoost captures additional fraud patterns that CatBoost may miss. This highlights that even high performing models are not identical in their decision boundaries. These differences can provide complementary signals for ensemble learning.

While not explicitly computed, we expect cross-family disagreement (neural vs tree) to be even higher due to fundamentally different learning mechanisms. All this is to suggest that an ensemble is promising as a meta learner might be able to exploit the disagreement regions beyond a simple weighted mixing.

4.5 Selection for Ensemble

Drawing together the insights from our individual model evaluations and disagreement analyses, we can make informed decisions for the final stacked ensemble architecture. XGBoost is selected as the primary foundational model of the ensemble due to its superior standalone performance, achieving the highest recall (0.7623) and PR-AUC (0.8019) across all evaluated methods. To complement it, CatBoost is also included. While XGBoost outperforms CatBoost in aggregate metrics, our earlier disagreement analysis revealed that the two models exhibit asymmetric fraud detection capabilities—with distinct decision boundaries and regions where each captures fraudulent transactions that the other misses. In contrast, LightGBM is excluded from the final selection. Despite providing a strong baseline, its inclusion offered diminishing returns and did not provide sufficient complementary value or distinct predictive power beyond the feature spaces already effectively covered by the combination of XGBoost and CatBoost. Finally, the Gated MLP is retained in the ensemble despite underperforming the tree-based models on raw aggregate metrics. As demonstrated by our earlier $\mathbf{D}_{\text{Rate}}$ analysis, the neural architecture captures fundamentally different, non-linear interactions within the anomaly feature space. By incorporating the Gated MLP, we inject necessary structural diversity into the ensemble. This allows the downstream meta-learner to consider a wider range of perspectives and exploit these high-disagreement regions to resolve ambiguous cases more effectively than a purely tree-based ensemble could.

5 Assembling the Ensemble

To maximize the predictive power of our models, we evaluated two distinct ensemble strategies: a manual Weighted Ensemble and a Stacked Ensemble (meta-learner). Our final ensemble incorporates a diverse set of five base learners, each capturing different representations of the data:

1. **Baseline Gated MLP:** Trained strictly on the raw, preprocessed tabular features.
2. **Gated MLP with AE-16:** Supplemented with a tightly compressed 16-dimensional Autoencoder latent space.
3. **Gated MLP with AE-256:** Supplemented with a wider 256-dimensional Autoencoder latent space.
4. **CatBoost with AE-256:** A gradient boosting tree incorporating the AE-256 reconstruction error metrics.
5. **XGBoost with AE-256:** Our strongest standalone tree model, also utilizing the AE-256 anomaly signals.

5.1 Weighted vs. Stacked Approach

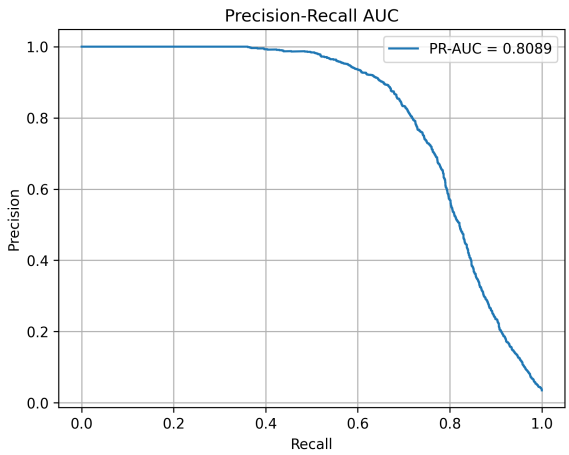
We initially conducted a grid search to find the optimal static weights for a `WeightedFraudEnsemble`. While this provided a strong baseline, static weighting limits the ensemble's ability to conditionally trust different models in different feature spaces. To overcome this, we implemented a `StackedFraudEnsemble`. We froze the weights of our five pre-trained base learners and fed their output probabilities (meta-features) into a newly initialised `StackingFraudMLP` which is a small neural network consisting of three hidden layers (32, 16, and 8 neurons) utilising GELU activations, Layer Normalization, and Dropout. The stacker was trained using the AdamW optimizer and a Binary Cross Entropy loss function weighted to penalize false negatives. As shown in Table 11, the Stacked Ensemble significantly outperformed the static weights, providing a massive boost to precision while maintaining high recall.

Ensemble Strategy	**F2**	**Precision**	**Recall**	**F1**
Manual Weighted	0.732	0.659	0.753	0.703
Stacked Neural Network	0.746	0.732	0.750	0.741

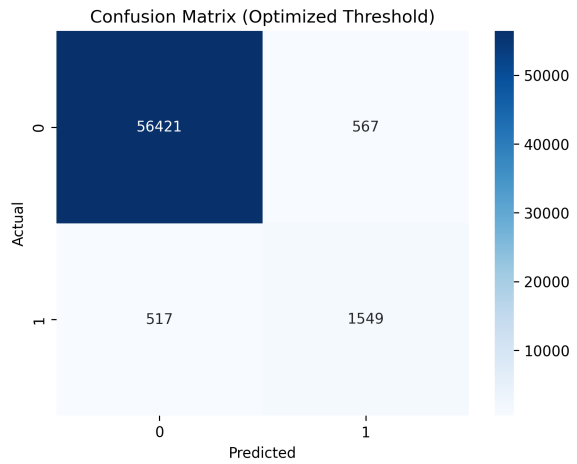
Table 11: Performance metrics of the ensemble strategies on the test set, optimized for the F2 score.

5.2 Stacker Probability Distribution and Results

An analysis of the stacker's final probability distribution on the test set demonstrates its effectiveness at class separation. The meta-learner successfully suppressed the probabilities of non-fraudulent transactions (pushing the mean negative probability down to just 0.012) while elevating fraudulent transactions (mean positive probability of 0.660). Because the stacker was highly confident in predicting non-fraud (resulting in a heavy left-skew for legitimate transactions), our dynamic threshold sweeping identified a lower optimal decision boundary. By adjusting the classification threshold from the default 0.50 down to 0.219, the ensemble minimized false negatives and achieved a final test F2-score of 0.746, proving the meta-learner's ability to synthesize both neural and tree-based probability outputs into a highly sensitive fraud detector.



(a) Precision–Recall curve (PR-AUC = 0.8089)



(b) Confusion matrix at F2-optimised threshold (0.2192)

Figure 11: Ensemble performance: ranking performance via PR curve and classification performance at the optimal decision threshold.

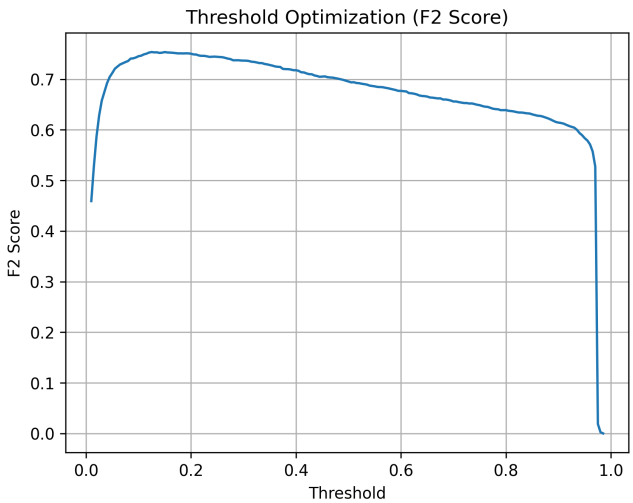


Figure 12: F2 score across classification thresholds for the Ensemble model

5.3 Class Separation and Probability Distribution

As illustrated in Figure 13, the meta-learner demonstrates a strong capability for class separation. By plotting the density of the stacker's output logits by class, we observe a massive, dense spike of non-fraudulent (normal) transactions clustered near zero probability. Conversely, the fraudulent transactions are distributed across a much wider, higher range of probabilities. This heavy left-skew for normal transactions mathematically explains our threshold-sweeping results: because the meta-learner is so confident about what constitutes a "normal" transaction, we can safely lower the classification threshold to **0.219** to catch more subtle fraud cases without triggering an overwhelming number of false positives.

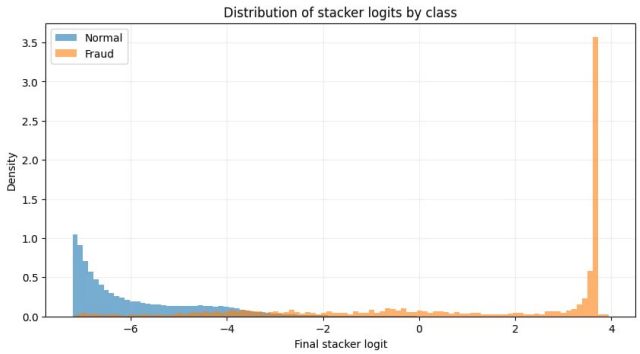


Figure 13: Distribution of Stacker Logits

5.4 System Architecture Diagram

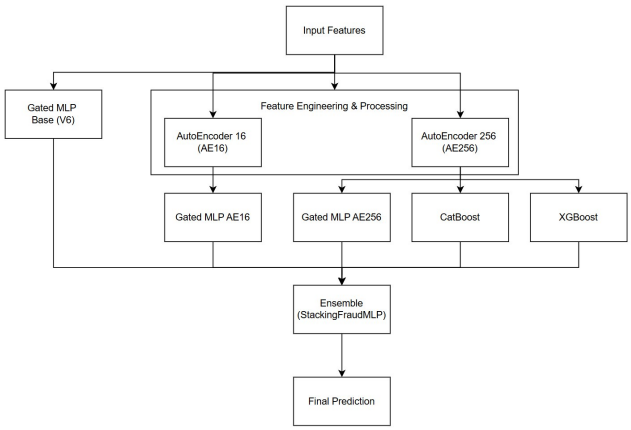
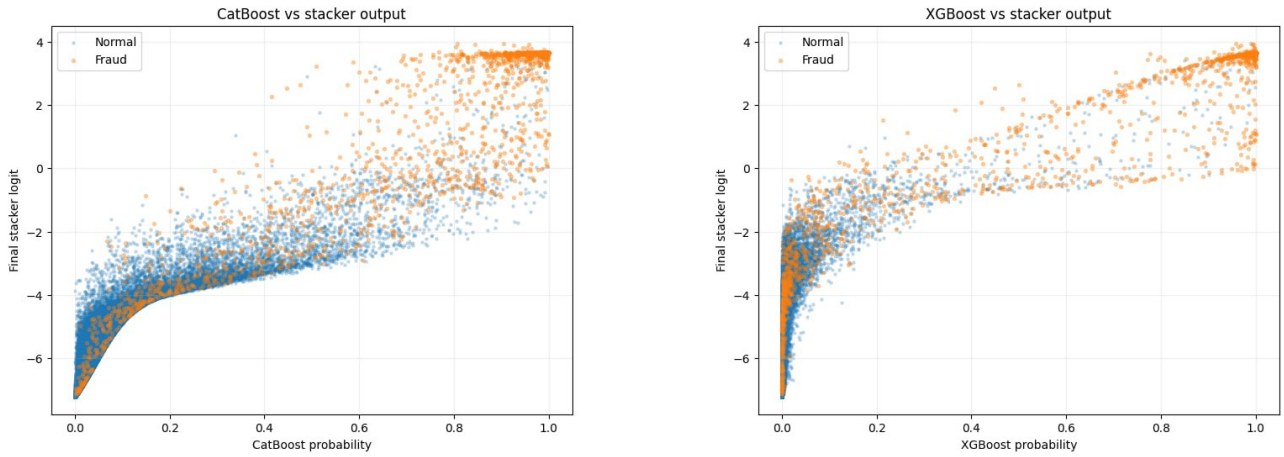


Figure 14: System Architecture Diagram

Our architecture uses a two-stage approach: a feature extraction layer using Standard AutoEncoders, followed by a heterogeneous ensemble whose outputs are optimised by a second-level MLP meta-learner to maximise the F2 score.

5.5 Base Learner Correlation

Furthermore, scatter plot analyses comparing the predictions of our strongest base learners (CatBoost and XGBoost) against the final Stacker output confirm that the meta-learner is not simply mirroring a single base model. Instead, it successfully learns a non-linear synthesis. For instances where CatBoost and XGBoost output moderate or conflicting probabilities, the meta-learner utilizes the signals from the Autoencoder-augmented MLP models to push the final prediction toward the correct class boundary.



(a) Catboost vs Stacker

(b) XGBoost vs Stacker

Figure 15: Comparison of Base Models against Neural Network Stacker

5.6 Error Distribution Analysis

Finally, to visualize the trade-offs at our optimized threshold of 0.219, we plotted the distribution of True Positives (TP), False Negatives (FN), True Negatives (TN), and False Positives (FP) (Figure 16).

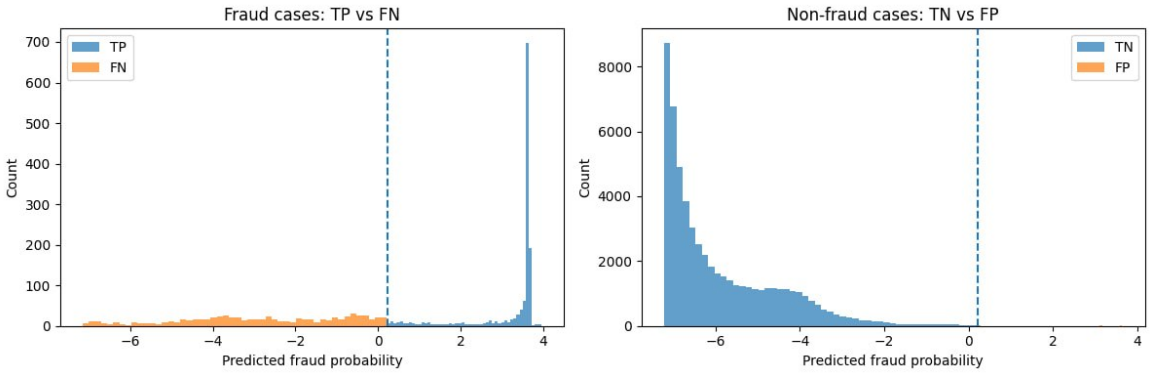


Figure 16: Distribution of classification errors (TP vs FN, and TN vs FP)

The False Negative distribution reveals the inherent limitations of the dataset; a subset of fraudulent transactions perfectly mimics legitimate consumer behavior, resulting in predicted probabilities well below our decision boundary. However, the False Positive distribution shows that the trade-off is well-managed.

Here, there exists no threshold that can be optimized in order to increase the Recall and F2 without sacrificing significant precision or accuracy. It can be noted that model tolerates a controlled volume of false alarms—transactions that appear statistically anomalous but are ultimately legitimate—in order to maximize the capture of actual financial fraud, perfectly aligning with our goal of optimizing the F2-score.

6 SOTA Comparisons

Since the dataset originated from a Kaggle competition, we were unable to obtain the ground truth labels for the official test set, nor the individual metric scores of the top-performing models on the private leaderboard. Nevertheless, we obtained a submission score of 88.7 using our ensemble model. This suggests that our ensemble approach achieves strong predictive performance on unseen data. Top-performing models on the leaderboard typically conducted more extensive feature engineering and domain-specific analysis of transaction behaviour. In contrast, our approach prioritised model-driven representation learning through ensemble methods, achieving competitive performance without extensive manual feature engineering.

7 Conclusion

In conclusion, our methodology demonstrated that sophisticated feature engineering is just as critical as model selection. By moving beyond simple imputation and deploying a Denoising Autoencoder—trained exclusively on legitimate transactions, we successfully extracted powerful anomaly signals (such as reconstruction errors and latent embeddings). These augmented features provided the downstream classifiers with a significantly stronger mathematical basis for isolating fraudulent patterns. Through rigorous evaluation, we found that sequence-aware models like Transformers struggled due to the noisy, heuristic grouping of user identities in anonymized data. Instead, point-based tabular methods proved far more effective. Our custom Gated MLP successfully bridged the performance gap traditionally seen between neural networks and tree-based models on tabular data, while XGBoost emerged as the strongest standalone classifier, adeptly handling the sparsity of the dataset. Ultimately, we achieved our best, most balanced performance by synthesizing these diverse approaches into a Stacked Neural Network Ensemble. By feeding the probabilistic outputs of five distinct base learners (incorporating both raw features and Autoencoder augmentations) into a meta-learner, the ensemble achieved a test F2-score of 0.746. Our analysis of the stacker's logits revealed a profound ability to compress the probability space of normal transactions near zero. This allowed us to dynamically lower the classification threshold to 0.219, maximizing the model's sensitivity to fraud while maintaining a highly controlled, acceptable rate of false positives. In summary, this project highlights the efficacy of combining representation learning (via Autoencoders) with the predictive power of gradient boosting and neural network stacking. The resulting architecture provides a highly sensitive, interpretable, and scalable solution for real-world financial fraud detection.

8 Task Distribution

Student Name	**Task Distribution**
Lee Sean	Report writing and analysis. Model exploration and data processing.
Meng Teck	Report writing, model architecture and model training.
Selina Tan Hui Shan	Report writing, model exploration, model training and research.

9 Load Models

In order to load the models, we have provided a file called `ModelLoadingExample.ipynb` that has all the loading functions implemented in the ipynb cells in order to fully load the completed ensemble.

10 GitHub Link

Project code available at: `https://github.com/leesean5150/DL_project.git`

11 References

- Aminian, G., Elliott, A., Li, T., Wong, T. C. H., Dehon, V. C., Szpruch, L., Maple, C., Read, C., Brown, M., Reinert, G., & Mamouei, M. (2025). FraudTransformer: Time-Aware GPT for Transaction Fraud Detection. arXiv. https://arxiv.org/abs/2509.23712
- IEEE Computational Intelligence Society. (2019). IEEE-CIS fraud detection dataset [Data set]. Kaggle. https://www.kaggle.com/competitions/ieee-fraud-detection